



Pokerbots 2026

Lecture 10: How to Write Performant Code (Part 1)

Sponsors



CITADEL | CITADEL Securities



New Sponsor Alert!?



CITADEL | CITADEL Securities



Sponsors



CITADEL | CITADEL Securities





Announcements

HACKATHON PART 2 (TODAY)

- Students expressed interest at pkr.bot/hack
- TODAY, 7pm – LATE (32-044)
 - However late it ends up going
- Awesome food/drinks
- *More prizes/games*
- See you there!



Week 3 Bot Deadline

- Third pokerbot submission due Friday 1/23, 11:59PM EST on scrimmage server
- Mini-tournament 3 will occur shortly after
- Good luck!



Next Week...

Monday 1/26

Lecture 11 (part 2 of this lecture)

Tuesday 1/27

Team Strategy Reports due 11:59PM EST

- No Lecture
- 3-5 pages, double spaced, ≥ 1 member must submit
- Assignment details and submission will be on Canvas: pkr.bot/canvas
- Description also available in syllabus: pkr.bot/syllabus

Wednesday 1/28

Guest Lecture: Dr. Noam Brown

- Won Pokerbots and co-created Deep CFR while PhD at CMU
- Created Libratus and Pluribus, the world's first superhuman poker AIs
- Research scientist currently at OpenAI, previously worked at Meta AI
- Leading role in developing OpenAI's latest GPT o1 and o3 LLMs

Pinned

Noam Brown @polynoamial · Sep 12, 2024

Today, I'm excited to share with you all the fruit of our effort at @OpenAI to create AI models capable of truly general reasoning: OpenAI's new o1 model series! (aka 🧠) Let me explain 📊 1/

Benchmark	Model	Accuracy (%)
Competition Math (AIME 2024)	gpt4o	13.4
	o1 preview	56.7
	o1	83.3
Competition Code (CodeForces)	gpt4o	11.0
	o1 preview	62.0
	o1	89.0
PhD-Level Science Questions (GPQA Diamond)	gpt4o	56.1
	o1 preview	78.3
	o1	78.0
	expert human	69.7

222 1.9K 11K 2.4M



Wednesday 1/28 (cont.)

Final Bot Submission due 11:59PM EST

- Upload and select bot as active on scrimmage server
- Both report and bot needed to pass this class!
- Bot will compete in last and final Pokerbots tournament
- Non-secret prize amounts listed on syllabus

Final Tournament Prizes	
First place	\$10,000
Second place	\$6,500
Third place	\$3,500
Fourth place	\$2,000
Fifth place	\$1,000
First place in language (Python, Java, or C++)	\$500 x 3
Second place in language (Python, Java, or C++)	\$250 x 3
Third place in language (Python, Java, or C++)	\$125 x 3
Best freshman-majority (>51%) team	\$2,000

Friday 1/30

Pokerbots Final Event 4:30-7PM in Tang Center (E51)

- Presentation of Awards
- Closing Ceremony
- Sponsor Event and Puzzle Hunt
- Lots of free merch and raffle prizes!
- Dinner provided 😊

Sat 1/31

Epic Poker Tournament!

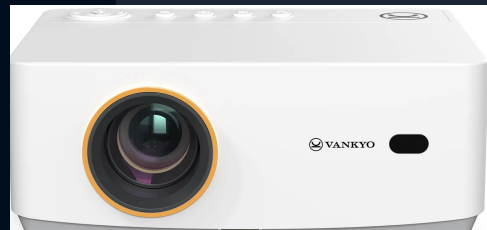
- 5-8PM in BC Porter Room (tentative)
- \$3500 cash prize pool



Giveaway!

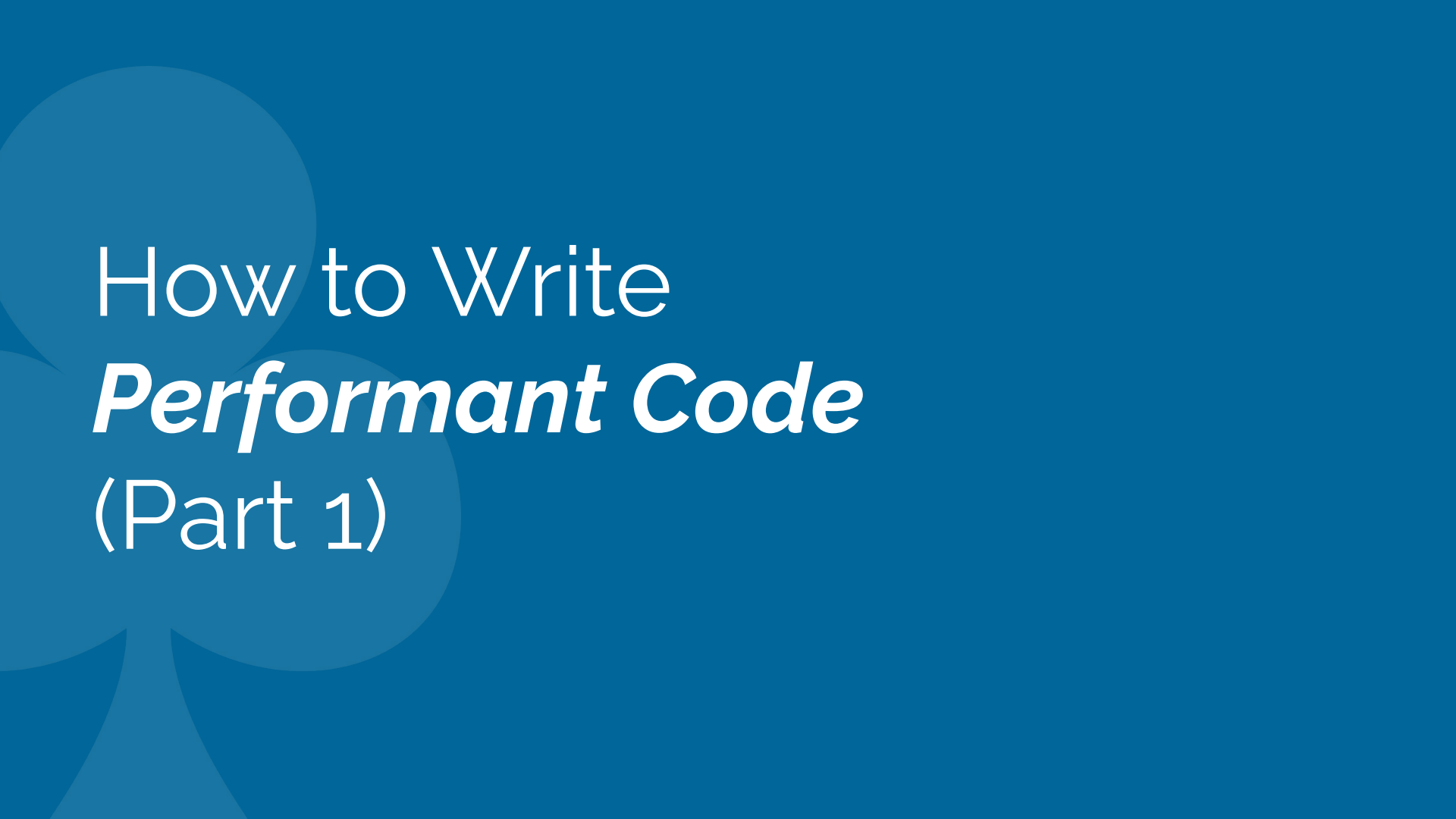
Seven Game

- Submit a permutation of the digits from 0 to 7 (i.e. 01234567)
 - Invalid submissions will be disregarded
- $\text{Distance}(a1, a2) = \# \text{ of positions at which digits differ (i.e. hamming distance)}$
 - e.g. $\text{distance}(0123, 1023) = 2$
- Goal: Maximize average distance from everyone else!
 - (Ties broken by earliest submission)
- Prize: Vankyo Leisure 470W Projector



Questions?

pkr.bot/sev

The background is a solid blue color with several overlapping circles of varying shades of blue, creating a layered effect. The text is white and positioned on the left side of the image.

How to Write *Performant Code* (Part 1)

Agenda

- How Do We Measure Speed?
- Big-O Notation
- The Constant Failures of the Theoretical
- C++ vs. Java vs. Python
- What Next?



How Do We Measure Speed?

Speed = Time

1. Write code
2. Start timer
3. Run code
4. Stop timer
5. Jot down time

Issue: human lag leads to biased measurements

Solution?



C o m p u t e r s

PROFILING TIME!!!

Profiling = the process of analyzing a program's runtime behavior to measure resource usage and identify performance bottlenecks

PROFILING TIME!!!

Profiling = the process of analyzing a program's runtime behavior to measure resource usage and identify performance bottlenecks

PROFILING TIME!!!

Profiling = the process of analyzing a program's runtime behavior to measure resource usage and identify performance bottlenecks

PROFILING TIME!!!

Profiling = the process of analyzing a program's runtime behavior to measure resource usage and identify performance bottlenecks

*well bobby, profiling sounds **AWESOME**, how can I profile my little python program?*

```
set -euo pipefail
```

```
# "why is it slow" starter pack
export PYTHONUNBUFFERED=1
export PYTHONDONTWRITEBYTECODE=1
export PYTHONHASHSEED=0
export OMP_NUM_THREADS=1
export MKL_NUM_THREADS=1
export OPENBLAS_NUM_THREADS=1
export VECLIB_MAXIMUM_THREADS=1
export NUMEXPR_NUM_THREADS=1
```

```
RUN_ID="$(date +%Y%m%d_%H%M%S)"
OUT="prof_${RUN_ID}"
mkdir -p "$OUT"
```

```
echo "[1/7] sanity"
python -V | tee "$OUT/python_version.txt"
uname -a | tee "$OUT/uname.txt" || true
```

```
echo "[2/7] warmup run (JIT/caches/imports/etc)"
taskset -c 2 python -X faulthandler -X importtime script.py \
  --mode warmup --steps 50 \
  2> "$OUT/importtime.log" || true
```

```
echo "[3/7] sampling profiler (py-spy)"
python -u script.py --mode run --steps 999999 --log debug >"$OUT/run.log" 2>&1 &
PID=$!
echo "PID=$PID" | tee "$OUT/pid.txt"
sleep 1
```

```
py-spy record --pid "$PID" --rate 999 -o "$OUT/flame.svg" --duration 15 || true
py-spy dump --pid "$PID" --native --threads > "$OUT/stack_dump.txt" || true
```

```
echo "[4/7] perf stat (hardware counters)"
# (may require permissions; we're being dramatic)
perf stat -ddd \
  -e cycles,instructions,branches,branch-misses \
  -e cache-references,cache-misses \
  -e L1-dcache-loads,L1-dcache-load-misses \
  -e dTLB-loads,dTLB-load-misses \
  -e context-switches,cpu-migrations,page-faults \
  -p "$PID" -- sleep 10 \
  2> "$OUT/perf_stat.txt" || true
```

```
echo "[5/7] perf record (call stacks)"
perf record -F 999 -g -p "$PID" -- sleep 10 \
  -o "$OUT/perf.data" || true
perf report --stdio -i "$OUT/perf.data" > "$OUT/perf_report.txt" || true
```

```
echo "[6/7] grab last logs + stop"
tail -n 200 "$OUT/run.log" > "$OUT/run_tail.log" || true
kill -INT "$PID" || true
wait "$PID" 2>/dev/null || true
```

```
echo "[7/7] artifacts"
ls -lh "$OUT" | tee "$OUT/ls.txt"
echo "open $OUT/flame.svg"
echo "read $OUT/perf_report.txt"
echo "then cry"
```

```
print("t = '
#
pyt to print
del blob
gc.collect()
print("\nafter gc current/peak:",
py-
ll -IN
t $PID
ll flame.sv
flame.svg
```



OK PAUSE

Phew... that was a lot



Big-O Notation

Let's Take a Step Back

- Profilers are a tough place to start
- What if instead of actually run the code, we run it in our head?

I don't know about you, but I can't do that



I don't know about you, but I can't do that

- But we don't need to truly run the code
- Just need a *vague idea* of the runtime's relationship to some parameters
 - Here, parameters = values in our algorithm which affect the runtime
- For this idea to work, first we can assume a few things:
 1. We run our code on some “processor”
 2. Our processor can execute “instructions” from a given set (i.e. addition, memory access, etc)
 3. Our code can easily be transformed into a set of these “instructions”
 4. The time it takes to execute each different “instruction” is roughly similar

Big-O Notation

- Given our assumptions, we can sort of *guess* information about code runtime
- Let's go over an example:

```
my_list = [6, 9, 6, 3, 0, 2]
sum = 0
for val in my_list:
    sum += val
print(sum)
```

variables?

which ones matter?

~len(my_list) instructions!

$O(n)$

Big-O Notation

Assumptions:

1. We run our code on some “processor”
2. Our processor can execute “instructions” from a given set (i.e. addition, memory access, etc)
3. Our code can easily be transformed into a set of these “instructions”
4. The time it takes to execute each different “instruction” is roughly similar

What is O?

An algorithm's runtime is $O(g(n))$ for some function g if there exists constants c and n_0 such that $0 \leq \text{runtime} \leq c \cdot g(n)$ for all $n \geq n_0$

This means that our program's runtime (which we can assume is proportional to the number of instructions the processor performs) grows no faster than $g(n)$ (up to a constant factor) for large enough inputs, providing an upper bound on its growth rate!

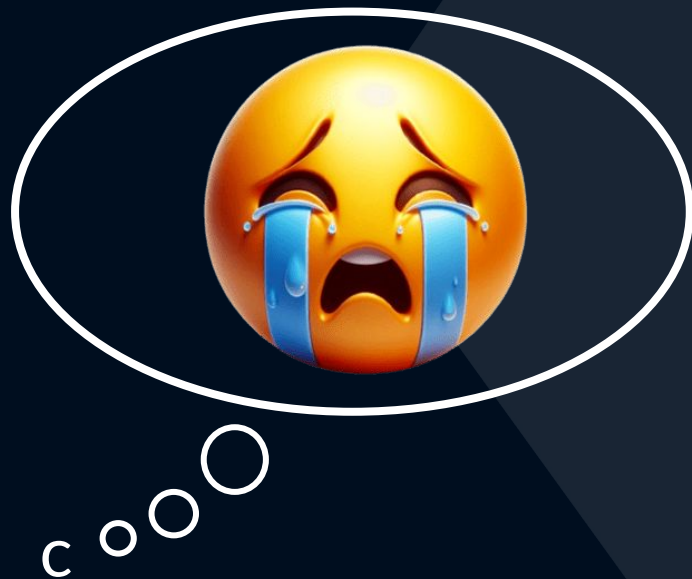
Big-O Notation

Essentially $O(n)$ means that the program's runtime $\propto n$

Number of instructions $\sim c \cdot n$

The constant factor is irrelevant in Big-O Notation!

Something else to notice is that because we want an upper bound for the runtime, we **generally** look at the worst possible case if our algorithm depends on variables which are too difficult to model (i.e. linear search)



Example Time!

```
print("hello world!")
```

What is the time complexity?

$O(1)$

Example Time!

```
print("6")  
print(".")  
print("9")  
print("6")  
print("3")  
print("0")
```

Constants don't matter!

6 prints of constant strings is still $O(1)$!

$O(1)$

What is the time complexity?

Example Time!

```
def naive_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(i + 1, n):  
            if arr[i] > arr[j]:  
                arr[i], arr[j] = arr[j], arr[i]
```

We loop through all n elements, and for each one, we loop through $n - i$ elements, getting us $1+2+\dots+n \sim n^2$!

$O(n^2)$

What is the time complexity?

Example Time!

```
def super_secret_sort(arr):  
    arr.sort() # ???
```

Using builtin functions is not free!

Python uses Timsort which combines merge and insertion sort and runs in $O(n \log n)$

What is the time complexity?

$O(n \log n)$

“if x in arr” runs a linear search through the array!

Example Time!

Each call takes $O(m)$ time!

```
# year1_ids and year2_ids are Python Lists
def count_students_who_took_pokerbots_in_both_years(year1_ids, year2_ids):
    n = len(year1_ids)
    m = len(year2_ids)
    count = 0
    for id in year1_ids:
        if id in year2_ids:
            count += 1
    return count
```

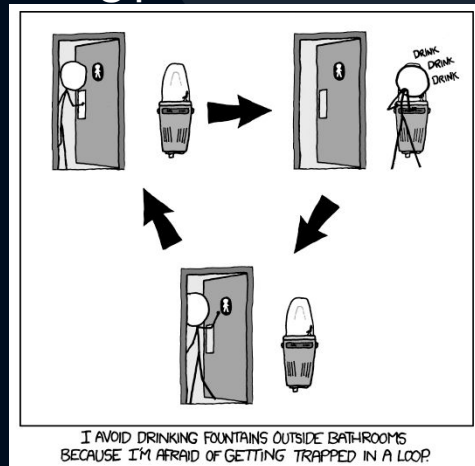
What is the time complexity?

$O(n \cdot m)$

Questions?

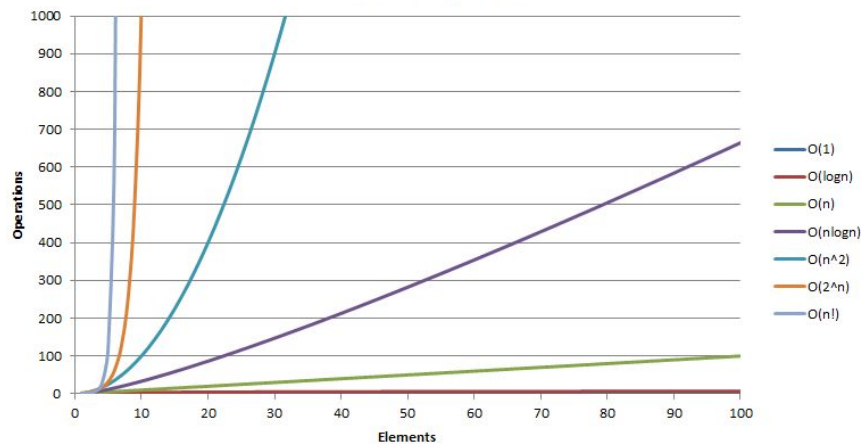
Halting Problem

- Can we write a program to determine time complexity of a code piece?
 - Unfortunately, no
- Halting Problem
 - Proven undecidable by Alan Turing
 - There is no program that can take any program + input and always decide whether it halts
- If we could figure out time complexity, we could solve the halting problem!
- Rice's Theorem
 - Any non-trivial semantic property of programs is undecidable
 - (Time complexity is a semantic property)



Tying It All Together

Big-O Complexity



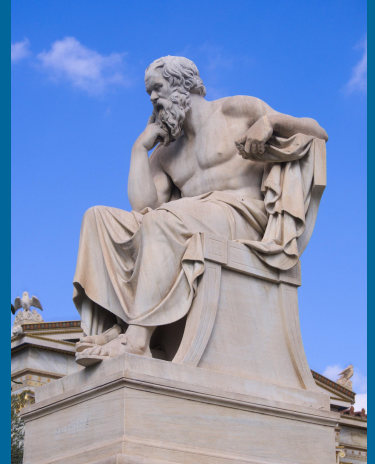
	<i>constant</i>	<i>logarithmic</i>	<i>linear</i>	<i>N-log-N</i>	<i>quadratic</i>	<i>cubic</i>	<i>exponential</i>
<i>n</i>	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256
16	1	4	16	64	256	4,096	65536
32	1	5	32	160	1,024	32,768	4,294,967,296
64	1	6	64	384	4,069	262,144	1.84 x 10 ¹⁹

Honorary mention: $O(\sqrt{n})$

Cool note: $O(2^{n/2})$ seems like $O(2^n)$ with a constant, but it is actually $O(\sqrt{2^n})$!

The Constant Failures of the Theoretical

How I felt
making that title



Constant Magic

Let's assume we are solving a linear search, where n is guaranteed $\leq 100,000$

```
def magic_search(arr, val):  
    count = 0  
    for i in range(100000):  
        if i < len(arr):  
            if arr[i] == val:  
                return i  
    return -1
```

What is the time complexity?

$O(1)???$

Constant Magic?

- Not fair to say it is $O(1)$
- Whole point of Big-O is to measure scaling w.r.t families of inputs
 - intrinsically assumes that inputs can grow arbitrarily large
- With n being capped to 100,000, we change the linear search problem's nature

but in practice, we almost always have input constraints

so why do we even care about Big-O?

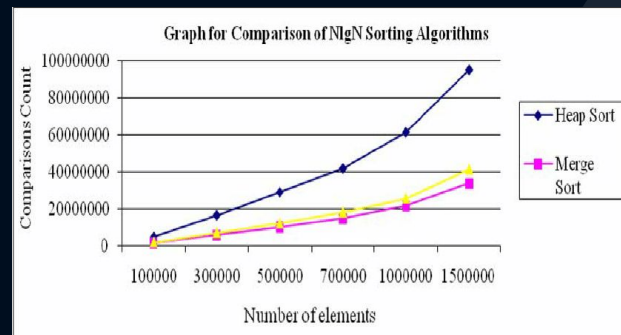


Constant Confusion

Furthermore, Big-O leaves out constants, which are super important in practice!

Examples:

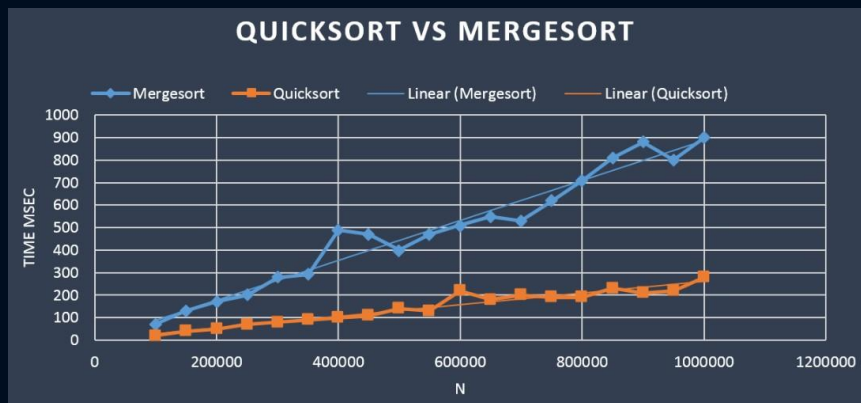
- `pkrbot.evaluate(hand)` runs in $O(1)$
 - so does `x += 1`
- Merge sort runs in $O(n \log n)$
 - so does Heapsort
 - so does Treesort



But these functions tend to have widely different performance in practice!

Constant Confusion!

Even worse, Quicksort, which literally has “quick” in the name, runs in $O(n^2)$ worst case!



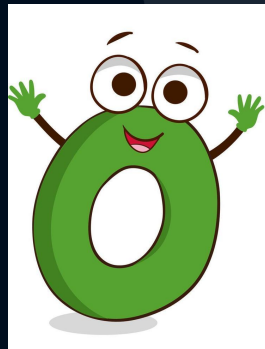
but in practice it's one of the fastest sorting algorithms one can use

so again, why do we even care about Big-O?



The Right Way to Use Big-O in Practice

- Despite its practical flaws, Big-O plays a major role in writing performant code
- Rather than using it as a tool to directly analyze runtime, we can use it as a nice abstraction while exploring approaches to a problem
- Divide large problems into recognizable subproblems
 - for each subproblem, we will have an idea of how the runtime of different approaches tends to scale with the inputs



The Right Way to Use Big-O in Practice

Example thought process:

Problem: given array of N numbers and Q search queries, return index in array or -1
(assume $N \sim Q$)

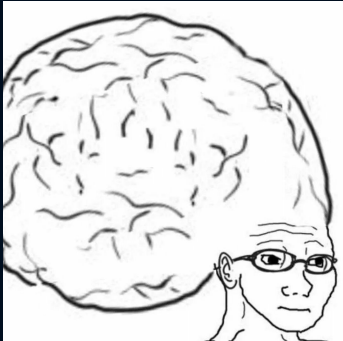
Approach: Q linear searches, runtime is $O(NQ)$

Goal: lower this theoretical runtime!

The Right Way to Use Big-O in Practice

Critical idea 1: because $N \sim Q$, then $O(NQ) \sim O(N^2)$, so adding anything with a lower complexity (i.e. $O(N)$ or $O(N \log N)$) to our algorithm cannot worsen our complexity!

Critical idea 2: we know binary search works in $O(\log N)$, but array is not sorted!



Big brain realization: sorting can be done in $O(N \log N)$, which is less than the complexity of our current approach, allowing us to run Q binary searches instead of linear searches!

Now our runtime becomes $O(N \log N + Q \log N) \sim \underline{O(N \log N)}$

Questions?

Lunch 🤪

Leave any type of feedback at [pkr.bot/feedback!](https://pkr.bot/feedback)

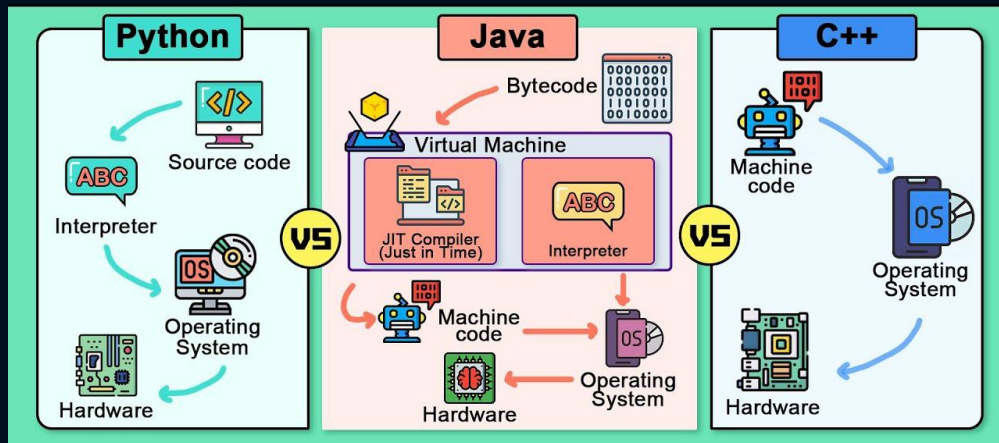


Language	Running time	Absolute Speed Up
Python	2821.108	1
Java	84.612	33.34
C	49.958	56.47

C++ vs. Java vs. Python

All Roads Lead to Machine Code

- Our CPUs can only run *machine code*
 - Made up of assembly instructions, a.k.a. the instruction set we discussed!
- Main difference in these languages is how exactly the human code gets **translated** into instructions



Compiled vs. Interpreted

- Python is interpreted (in practice, optimizations are made)
 - Each line that needs to get executed will get parsed and then executed during runtime
 - Super slow process with lots of overhead!
- Java and C++ are compiled
 - C++ is compiled directly into machine code, a.k.a. assembly instructions
 - Done just once, then original code never needs to be parsed again!
 - Low overhead
 - Java is compiled into **bytecode**, essentially platform-independent instructions
 - Done just once, then original code never needs to be parsed again!
 - However, bytecode is interpreted (in practice, optimized thoroughly), adding overhead!



=



Why to Choose Each

- Python
 - Easy to learn and set up
 - Libraries for ML
 - Slow in theory, but abstracted libraries are highly optimized!
- C++
 - Lower level control over memory
 - Intrinsic performance boost = more operations per second!
 - Extensive and optimized standard library
- Java
 - ...
 - (good if you already know it I guess)



What Next?

Practical Performance

- Today was mostly an introduction to performance as a concept, emphasizing the difference between theoretical and practical performance
- Next time, we will go much deeper into the practical side!
 - How processors work
 - Compute vs. memory
 - Cache locality
 - Relevant examples



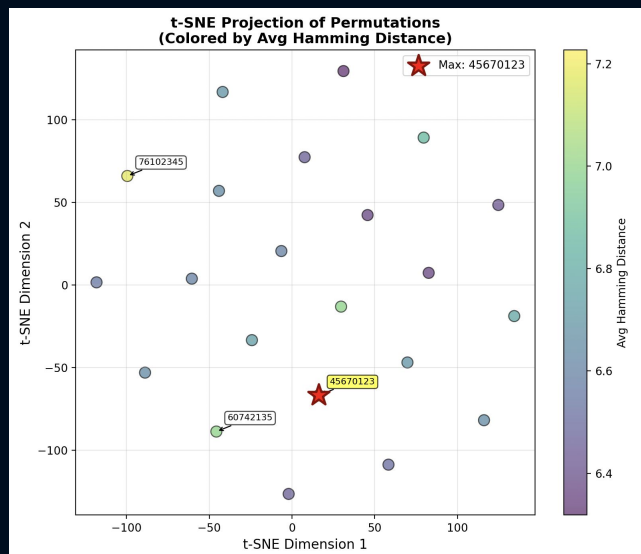
Giveaway Winner!

Seven Winner!

Winner avg dist: 7.23!

67 %: 36.4 (expected 12.5%)

~~We are doomed as a society~~



Winner:
aschwanz

Thanks for watching!

Information hub at pkr.bot/hub

Slides/notes will be posted on pkr.bot/resources

Make sure to check pkr.bot/piazza for updates

Lecture recordings at pkr.bot/panopto

Leave feedback at pkr.bot/feedback!